

HTTP Connections & Response Time Analysis

1 HTTP Connections Overview

HTTP connections are the foundation of web communication, determining how efficiently data is transferred between clients and servers. There are two primary types of HTTP connections, each with distinct characteristics and performance implications.

Key Concept: HTTP Connection Types

HTTP can establish connections in two ways:

- **Non-Persistent HTTP**: Each object requires a separate TCP connection
- **Persistent HTTP**: Multiple objects share a single TCP connection

2 Non-Persistent HTTP (HTTP/1.0)

2.1 Definition & Characteristics

Non-Persistent HTTP follows a simple but inefficient model where each web object requires its own dedicated TCP connection.

Non-Persistent HTTP Characteristics

- **One TCP connection per object**
- Connection is **closed** immediately after object transfer
- Multiple objects require multiple separate connections
- Default behavior in **HTTP/1.0**
- Higher overhead due to repeated connection setup/teardown

2.2 Operation Process

The non-persistent HTTP process follows a strict sequence of steps for each object:

Non-Persistent HTTP Steps

1. **TCP Connection Initiation:** HTTP client initiates TCP connection to server (typically port 80)
2. **Request Transmission:** Client sends HTTP request message containing the URL
3. **Server Processing:** Server accepts connection, processes request, and forms response
4. **Object Delivery:** Server sends requested object back to client
5. **Connection Termination:** Server immediately closes TCP connection
6. **Repeat Process:** For additional objects, the entire process repeats from step 1

2.3 Performance Issues

Non-Persistent HTTP Limitations

- **High RTT Overhead:** Requires 2 RTTs per object (setup + request/response)
- **OS Resource Waste:** Each TCP connection consumes system resources
- **Network Congestion:** Multiple parallel connections can overwhelm networks
- **Poor Scalability:** Performance degrades with increasing number of objects

Non-Persistent HTTP Total Time

For n objects: $T_{total} = n \times (2 \times RTT + T_{transmission})$

3 Persistent HTTP (HTTP/1.1+)

3.1 Definition & Advantages

Persistent HTTP revolutionizes web performance by maintaining open connections for multiple object transfers.

Persistent HTTP Characteristics

- **Single TCP connection for multiple objects**
- Connection remains **open** after object delivery
- Dramatically reduces connection overhead
- Default behavior in **HTTP/1.1** and later versions
- Enables efficient resource utilization

3.2 Performance Benefits

Key Advantages of Persistent HTTP

- **Reduced Latency**: Only 1 RTT needed for subsequent objects
- **Lower Resource Usage**: Single connection reduces OS overhead
- **Improved Throughput**: Better bandwidth utilization
- **Enhanced User Experience**: Faster page loading times

4 NonPersistent and Persistent HTTP Diagram

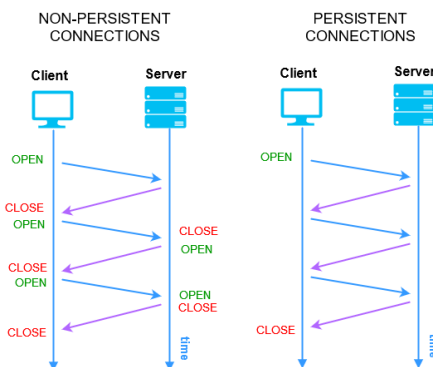


Figure 1: **Persistent vs Non-Persistent HTTP**: Non-persistent requires multiple TCP connections, while persistent uses a single maintained TCP connection.

5 HTTP Response Time Analysis

Understanding response time is crucial for optimizing web performance. Response time depends on network delays and connection strategies.

Round-Trip Time (RTT) - Foundation Concept

RTT = Time for a small packet to travel from client to server and back

RTT Components:

- **Processing delays** - Server processing time
- **Queueing delays** - Network congestion delays
- **Transmission delays** - Time to put packet on wire
- **Propagation delays** - Physical transmission time

5.1 Non-Persistent HTTP Response Time

For **each object** in non-persistent HTTP, the process requires exactly 2 RTTs plus transmission time.

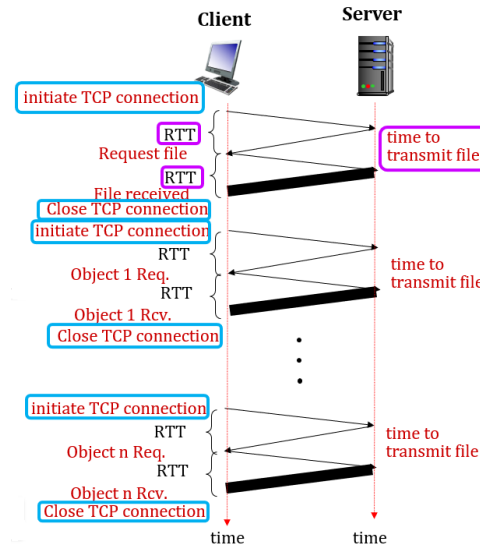


Figure 2: **Non-Persistent HTTP Timeline:** Detailed breakdown of response time components

Non-Persistent HTTP Response Time Formula

$$T_{response} = 2 \times RTT + T_{transmission}$$

Where:

- RTT = Round-trip time
- $T_{transmission}$ = File transmission time = $\frac{\text{File Size}}{\text{Bandwidth}}$

5.2 Persistent HTTP Response Time

For **all objects** in persistent HTTP, only **one TCP connection setup** is required. After that, each object requires just 1 RTT plus transmission time.

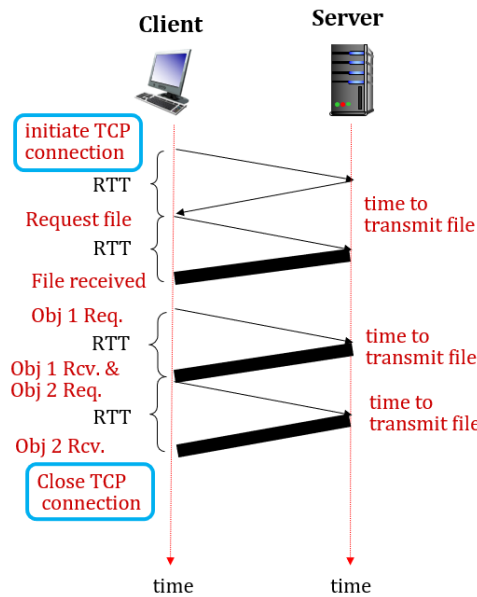


Figure 3: **Persistent HTTP Timeline:** Efficient reuse of TCP connection for multiple objects

Persistent HTTP Timing Analysis

First Object (includes initial TCP setup):

$$T_{first} = 2 \times RTT + T_{transmission}$$

Subsequent Objects (connection already established):

$$T_{additional} = 1 \times RTT + T_{transmission}$$

5.3 Performance Comparison Analysis

Let's analyze a realistic scenario to understand the performance impact:

Example Scenario Analysis

Scenario: Retrieve a web page with 1 HTML file + 10 images

Assumptions: Each object has same transmission time T_{file}

Connection Type	Setup RTTs	Data RTTs	Total Time
Non-Persistent HTTP	$11 \times 2 = 22$	0	$22 \times RTT + 11 \times T_{file}$
Persistent HTTP	$1 \times 2 = 2$	$10 \times 1 = 10$	$12 \times RTT + 11 \times T_{file}$
RTT Improvement	10 RTTs saved (45% reduction)		45% faster

Table 1: **Performance Comparison:** Non-Persistent vs Persistent HTTP for 11 objects

General Performance Formulas

Non-Persistent HTTP (n objects):

$$T_{\text{total}} = n \times (2 \cdot RTT + T_{\text{transmission}})$$

Persistent HTTP (n objects, single connection, sequential requests):

$$T_{\text{total}} = (n + 1) \cdot RTT + n \cdot T_{\text{transmission}}$$

RTT Savings with Persistent HTTP:

$$\Delta RTT = 2n - (n + 1) = n - 1 \quad \text{RTTs saved}$$

Key Performance Insights

- **Scaling Benefits:** As the number of objects increases, persistent HTTP advantages grow
- **Break-even Point:** Persistent HTTP is beneficial for $n \geq 2$ objects
- **Real-world Impact:** Modern web pages with 50+ objects see dramatic improvements
- **Network Efficiency:** Reduced connection overhead leads to better bandwidth utilization

6 Summary & Conclusions

Chapter Summary: HTTP Connections & Performance

Key Takeaways:

- **Non-Persistent HTTP** requires separate connections for each object, leading to high overhead
- **Persistent HTTP** maintains connections across multiple requests, dramatically improving efficiency
- **Performance improvement** with persistent HTTP increases with the number of objects per page
- **Real-world impact:** Modern web applications benefit significantly from persistent connections

Historical Context & Evolution

- **HTTP/1.0:** Non-persistent by default, leading to performance bottlenecks
- **HTTP/1.1:** Introduced persistent connections as the default behavior
- **Modern Web:** HTTP/2 and HTTP/3 build upon persistent connection concepts with further optimizations